Security testing with OWASP ZAP: interview questions, answered with this project

Questions about security testing for QA engineers and SDETs (DAST with OWASP ZAP, security headers, cookies, access control, CI integration) that come up in interviews. Each answer ends with an **In the project** pointer. Links go to github.com/niranjankagri/security-testing-zap.

Versions: OWASP ZAP 2.17.0 (Docker image), Node 24, TypeScript 6.0. OWASP Top 10 categories are from the 2021 list.

Contents

1. Security testing basics (Q1-Q10)
2. OWASP ZAP (Q11-Q24)
3. Headers and cookies (Q25-Q34)
4. Injection and cross-site scripting (Q35-Q41)
5. Access control and authentication (Q42-Q52)
6. CI, process and scenarios (Q53-Q66)

1. Security testing basics

Q1. What is the role of a QA engineer in security?

Not to replace security specialists, but to catch the common, known problems on every change: misconfigured headers, weak cookies, injection, broken access control. Automated checks in CI, security cases in test design (who may see what?), and clear reports to developers. Penetration tests and threat modelling stay with specialists.

In the project: a ZAP scan and access-control tests that run on every push.

Q2. SAST, DAST, SCA: what is the difference?

SAST reads the source code (finds, for example, string-built SQL). DAST attacks the running application from outside (finds what the deployed app really does). SCA checks dependencies for known vulnerabilities. They find different things and complement each other.

In the project: DAST with ZAP, plus API tests; the design doc lists SAST and SCA as the next layers.

Q3. What is the OWASP Top 10?

A ranking of the most common and critical web application risks, maintained by OWASP. In the 2021 list: A01 Broken Access Control, A02 Cryptographic Failures, A03 Injection, A04 Insecure Design, A05 Security Misconfiguration, A06 Vulnerable and Outdated Components, A07 Identification and Authentication Failures, A08 Software and Data Integrity Failures, A09 Security Logging and Monitoring Failures, A10 Server-Side Request Forgery.

In the project: each planted defect is tagged with its category (defects.ts).

Q4. Which Top 10 risks can a scanner find, and which not?

Misconfiguration (A05) and many injection problems (A03) are visible from outside, so scanners find them well. Broken access control (A01) needs to know who owns what, and insecure design needs understanding of the business, so scanners mostly miss them.

In the project: six defects found by the baseline scan, one by the full scan, three only by API tests.

Q5. What is a false positive in security scanning, and how do you handle it?

An alert for something that is not a real vulnerability. Verify it (reproduce the request), then either fix it anyway if it is cheap and hardens the app, or mark the rule IGNORE with a written reason. Never silence it without a reason, and never lower the whole policy.

In the project: "Bypassing 403" fired twice. The first time it exposed a real weakness (ambiguous paths accepted), fixed. The second time it was a false positive (a 403 from the token check made ZAP treat the public login page as protected); the status changed to 400 and the rule stayed FAIL.

Q6. And a false negative?

A real vulnerability the scanner did not report. The reason for planted defects: if a known problem is not reported, the scan setup (spider, rules, authentication) is broken.

In the project: the verify step fails if any planted defect is missed.

Q7. Why should you never scan a system you do not own?

Active scans send attacks; without permission that is illegal in most countries and can damage systems. Scan your own test environments, with written permission, and never production without an agreement.

In the project: the app is scanned on a private Docker network that nothing else can reach.

Q8. What is a vulnerable-by-design app, and why use one?

An app with known vulnerabilities (OWASP Juice Shop, DVWA, WebGoat) for training and for testing tools. Its value for a test setup: you know what must be found.

In the project: NoteVault has a hardened and a vulnerable build of the same code.

Q9. How do you write security test cases as a tester?

From the data and roles: for every object, who may read, change, delete it? Then try it as the wrong user, without login, with changed ids and extra fields. From inputs: what happens with HTML, script, quotes, very long values? From flows: can steps be skipped or repeated (brute force)?

In the project: checks.ts does exactly that for notes and sign-in.

Q10. How do you report a security bug?

Privately (not in a public tracker), with the exact request and response, the impact in plain words ("any signed-in user can read any note"), the affected versions, and a suggested fix. Severity from impact and likelihood (CVSS if the team uses it).

In the project: the defect stories in the catalogue are written as impact statements.

2. OWASP ZAP

Q11. What is OWASP ZAP?

The Zed Attack Proxy, a free and open-source DAST tool. It sits between client and app as a proxy, spiders the site, checks every response passively and can attack it actively. It runs as a desktop app, headless in Docker, and through an API.

In the project: the official Docker image, pinned to 2.17.0.

Q12. What is the difference between the baseline scan and the full scan?

The baseline scan spiders the site and only looks at the responses (passive): headers, cookies, information leaks. It is safe and fast. The full scan adds the active scan: it sends attacks (cross-site scripting, SQL injection, path traversal) and needs more time and a disposable environment.

In the project: both, for both builds; the reflected XSS is only found by the full scan.

Q13. What does the spider do?

It discovers pages by following links and forms, starting from the target URL. Pages it cannot reach (behind login, only reachable by JavaScript) are not scanned unless you add authentication, the AJAX spider or an API definition.

In the project: the home page links the search with an example term, so the spider finds the vulnerable parameter; signed-in pages are left to the API tests (a limitation the README states).

Q14. How do you configure which alerts fail a build?

A rules file (`-c rules.tsv`): one line per rule id with FAIL, WARN, IGNORE or INFO. `zap-baseline.py -g` generates the full list with every rule on WARN as a starting point.

In the project: zap/rules.tsv: 142 rules, 16 FAIL, 2 IGNORE with reasons.

Q15. Why not rely on ZAP's exit code?

It says "fail", "warn" or "pass" for one scan. The decisions here need more: the hardened build must have no unreviewed warnings, and the vulnerable build must have specific alerts. A small script that reads the JSON reports makes both decisions and explains them.

In the project: verify.ts.

Q16. What do ZAP's risk and confidence mean?

Risk is the impact if the alert is real (Informational, Low, Medium, High). Confidence is how sure ZAP is that it is real (Low, Medium, High, Confirmed). A high-risk, low-confidence alert needs manual verification first.

In the project: the report parser keeps the risk; informational alerts only count when their rule is FAIL.

Q17. Why treat informational alerts differently?

They are notes ("this response looks like session management"), not problems. ZAP does not count them as warnings either. Failing on them would make the build red for nothing and teach people to ignore it.

In the project: a unit test covers the rule: informational WARN passes, informational FAIL does not.

Q18. How do you run ZAP in Docker against an app on your machine?

Either let ZAP reach the host (`host.docker.internal`, on Linux with `--add-host=host.docker.internal:host-gateway`), or, better, run the app in a container on the same Docker network and scan it by container name.

In the project: the second way; the vulnerable build is never published on a host port.

Q19. Why pin the ZAP version?

New versions add and change rules. A new alert should arrive in a deliberate update, not in an unrelated pull request that suddenly fails.

In the project: `ghcr.io/zaproxy/zaproxy:2.17.0` in scan.ts.

Q20. How long does a full scan take, and how do you keep it in CI?

It depends on the number of pages and parameters. Here about four minutes per build. Options: scan builds in parallel jobs, limit spider time, run the full scan nightly and the baseline on every push, or scan only changed areas.

In the project: two parallel jobs, one per build, each with baseline and full scan.

Q21. How do you scan pages behind a login?

A ZAP context with the login URL, the credentials of a test user, and a logged-in/logged-out indicator, or a script for complex logins; the scan runs as that user. Keep a separate test user per scan, since active scans change data.

In the project: not done; the access-control tests cover the signed-in part. Listed under "what a real setup adds".

Q22. How do you scan a REST API with ZAP?

`zap-api-scan.py` with an OpenAPI, SOAP or GraphQL definition: ZAP imports every endpoint and parameter instead of spidering, then scans them actively.

In the project: a next step; the API here is tested by hand-written tests instead.

Q23. What does the HTML report contain?

Every alert with description, risk, confidence, the affected URLs, the evidence (the part of the response that triggered it), the attack used, a solution, and references (CWE, WASC).

In the project: uploaded per build as a CI artifact; the summary table is on the run page.

Q24. Is a clean ZAP scan a security sign-off?

No. It means the rules ZAP has found nothing on the pages it reached. Access control, business logic, authenticated areas and design flaws need other tests, and a penetration test by specialists before major releases.

In the project: three of ten planted defects are invisible to ZAP.

3. Headers and cookies

Q25. What does a Content-Security-Policy do?

It tells the browser which sources of scripts, styles, images, frames and form targets are allowed. Even if an attacker injects a `<script>`, an inline or foreign script does not run. It limits the damage of XSS; it does not replace escaping.

In the project: `default-src 'self'` with no inline scripts; the `no-csp` defect removes it (ZAP 10038).

Q26. What is clickjacking, and how is it prevented?

A malicious site loads your page in an invisible frame and tricks the user into clicking on it. Prevented by `Content-Security-Policy: frame-ancestors 'none'` (modern) and `X-Frame-Options: DENY` (older browsers).

In the project: both headers; the defect removes them (ZAP 10020).

Q27. What does `X-Content-Type-Options: nosniff` do?

It stops browsers from guessing a content type different from the declared one, so an uploaded text file cannot be executed as script or style.

In the project: the `no-nosniff` defect (ZAP 10021).

Q28. Why is an `X-Powered-By` header a finding?

It names the technology and version, which tells an attacker which known vulnerabilities to try. Hiding it is not security by itself, but there is no reason to advertise.

In the project: the `powered-by-header` defect (ZAP 10037).

Q29. What do the cookie flags HttpOnly, Secure and SameSite do?

HttpOnly: scripts cannot read the cookie, so XSS cannot steal the session. Secure: sent over HTTPS only. SameSite: not sent (Strict) or only sent on top-level navigation (Lax) with requests from other sites, which blocks most CSRF.

In the project: all three on the session cookie; two defects remove HttpOnly and SameSite (ZAP 10010, 10054).

Q30. What is CSRF and how is it prevented?

Cross-site request forgery: another site makes the user's browser send a request with the user's cookies. Prevented by an anti-CSRF token in forms, SameSite cookies, and for APIs by requiring a content type a form cannot send (JSON).

In the project: a token in the login form (a test sends a forged one and gets 400), SameSite=Strict, JSON-only updates (415 otherwise).

Q31. What is session fixation?

The attacker gets a session id, makes the victim use it, the victim logs in, and the attacker's id is now signed in. Prevented by issuing a new session id at login.

In the project: a test checks that the session id changes at sign-in.

Q32. What do Referrer-Policy and Permissions-Policy do?

Referrer-Policy controls how much of the URL is sent to other sites (no-referrer: nothing). Permissions-Policy switches off browser features the app does not need (camera, microphone, location).

In the project: both set in the hardened build.

Q33. What is Cache-Control: no-store for, and why does ZAP report it?

It keeps private pages out of browser and proxy caches (a shared computer, a proxy log). ZAP reports "Non-Storable Content" as information, because it can matter for performance. Here it is intended.

In the project: IGNORE with that reason in the rules file.

Q34. Why is HSTS not checked here?

Strict-Transport-Security only applies over HTTPS, and the scan runs over HTTP inside a private network. In a deployed environment with TLS it would be a FAIL rule.

In the project: listed under "what a real setup adds".

4. Injection and cross-site scripting

Q35. What is reflected XSS?

Input from the request is written into the page without escaping, so a crafted link runs the attacker's script in the victim's session. Stored XSS saves the input and shows it to others; DOM XSS happens in client-side code.

In the project: the search term in the vulnerable build (ZAP 40012).

Q36. How do you prevent XSS?

Escape output for its context (HTML text, attribute, JavaScript, URL), use templating that escapes by default, avoid inserting HTML from strings in the browser, and add a CSP as a second line of defence.

In the project: `escapeHtml` on every value; a test checks the search term comes back escaped.

Q37. How does ZAP find reflected XSS?

The active scan sends payloads in every parameter and checks whether they come back unescaped in a place where they would run.

In the project: found with 1 instance on `/search?q=` in the vulnerable build, none in the hardened one.

Q38. Why did ZAP also report DOM-based XSS on the vulnerable build?

The DOM XSS rule loads pages in a browser with payloads and watches for execution. Because the reflected payload really executes in the page, that rule fires too. One defect, two rules, both pointing to the same parameter.

In the project: the detection requires 40012; 40026 appears alongside.

Q39. How would you test for SQL injection?

Quotes and SQL fragments in every parameter, watching for errors, changed results or timing differences; tools like ZAP's active scan or sqlmap. Prevention: parameterised queries, never string-built SQL.

In the project: no database, but SQL injection is a FAIL rule in the full scan.

Q40. What is the difference between validation and escaping?

Validation rejects input that does not fit (an id must be a number). Escaping makes output safe for where it goes. You need both, because valid input (a name like O'Brien) can still break output.

In the project: the API only matches numeric ids; the pages escape everything.

Q41. Why does the 500 page say only "Something went wrong"?

Stack traces and library names help attackers. Details go to the server log; the user gets a generic message.

In the project: the server's error handler.

5. Access control and authentication

Q42. What is IDOR (broken object level authorization)?

The app trusts an id from the request and does not check that the object belongs to the user: change `/api/notes/1` to `/api/notes/3` and read someone else's data. The most common serious API vulnerability.

In the project: the `idor-read-other-notes` defect; caught by `idorFindings`.

Q43. Why can't ZAP find IDOR?

It does not know that note 3 belongs to bob. Both answers (200 with a note) look normal. Only a test that knows the owners can say the answer is wrong.

In the project: the reason the API tests exist next to the scan.

Q44. Should the API answer 403 or 404 for someone else's object?

404 is often better: 403 confirms that the id exists, which helps attackers enumerate. Be consistent with how non-existing ids are answered.

In the project: 404, with a comment explaining why.

Q45. What is mass assignment?

The API copies all fields of the request into the object, including fields the user must not set (owner, role, price). Prevented by an allow-list of editable fields.

In the project: `EDITABLE = ['title', 'text']`; the defect accepts `ownerId`.

Q46. How do you test brute-force protection?

Send more wrong passwords than the limit allows and expect a refusal (429) at the right attempt, not later; check that the lock lasts and that it does not reveal which users exist.

In the project: `rateLimitFindings` expects 429 on the sixth attempt.

Q47. Why the same message for an unknown user and a wrong password?

Different messages let an attacker find valid user names. The same message and the same status for both.

In the project: a test compares status and message for both cases.

Q48. How should passwords be stored?

Never in plain text or with a fast hash. A slow, salted password hash (scrypt, Argon2, bcrypt), compared in constant time.

In the project: scrypt with a random salt per user, `timingSafeEqual`.

Q49. Why test access control with a fresh app per test?

Tests change state: the brute-force test locks a user. Shared state makes tests depend on order. A fresh server per test on a free port keeps them independent.

In the project: `start(build)` in security.test.ts.

Q50. Why do the access-control checks return findings instead of asserting?

The same check runs against both builds: empty on the hardened one, a specific finding on the vulnerable one. One implementation proven both ways.

In the project: every function in checks.ts.

Q51. What would you add for a real authentication system?

Multi-factor authentication, password reset flows (token expiry, no user enumeration), session timeout and logout on the server side, account lockout notifications, and logging of security events.

In the project: out of scope for the demo, stated in the design doc.

Q52. Why does the app refuse `/%2e/login` with 400?

The URL parser turns it into `/login`. If access rules were checked on the raw path and routing on the normalised one, `/%2e/admin` could bypass a rule for `/admin`. Refusing non-canonical paths removes the ambiguity.

In the project: found by ZAP's "Bypassing 403" rule; a test checks three ambiguous paths.

6. CI, process and scenarios

Q53. How do you add ZAP to a CI pipeline?

Start the app (or deploy to a test environment), run the baseline scan with a rules file, publish the report, and fail on FAIL rules. Add the full scan where time allows (parallel jobs or nightly). Start with WARN everywhere, then tighten.

In the project: ci.yml: checks → scans in parallel → verify.

Q54. How do you introduce scanning in a project with many existing alerts?

Run once, triage every alert: fix, IGNORE with a reason, or keep as WARN with a ticket. Set FAIL only for rules that are clean now, so new problems fail the build without blocking on old ones. Burn down the rest.

In the project: the rules file shows the end state: FAIL for the important rules, two documented IGNOREs.

Q55. How do you prove your security tests work?

Plant known vulnerabilities in a copy of the app and require each to be reported by the expected rule or test. If the spider misses a page, a rule is disabled, or authentication breaks, the proof fails.

In the project: ten defects, each with a named detection, judged in CI.

Q56. How are planted defects kept from disappearing?

A unit test checks that every defect id still appears in the server code as `has('<id>')`, and that every ZAP detection's rule is FAIL in the rules file.

In the project: unit.test.ts.

Q57. What did the scan find in your hardened build?

Two things: the error page after a failed CSRF check showed a login form with an empty token, and an encoded dot in the path was accepted. Both fixed; both are now covered by tests.

In the project: section 7 of the design doc.

Q58. A scan reports a new medium alert on the main branch. What do you do?

Reproduce it from the report's request and evidence, decide whether it is real, find the change that introduced it, then fix or document. If it comes from a ZAP upgrade, handle it in that upgrade's pull request.

In the project: a new WARN on the hardened build fails verify, so this conversation happens.

Q59. A developer says "the header is set by the proxy in production, ignore it". What do you answer?

Then scan an environment with the same proxy, or set the header in the app too (defence in depth). An IGNORE with that reason is acceptable only if production is checked somewhere.

In the project: HSTS is the same case: listed for the deployed environment.

Q60. How do you keep the deliberately vulnerable build safe?

Never deploy it; run it only on an isolated network; mark it clearly in code and docs; no real data in it.

In the project: private Docker network, read-only mount, a warning in defects.ts.

Q61. Why is the app in the repository instead of a hosted test site?

The scan needs a target I am allowed to attack, that does not change by itself, and whose defects are known. Each run starts fresh copies.

In the project: the README's "Why the app is in this repository".

Q62. How would you scan a single-page application?

Use the AJAX or client spider (a real browser), scan the API behind it with its OpenAPI definition, and add a context for authentication.

In the project: server-rendered pages, so the traditional spider is enough.

Q63. What other tools would you put next to ZAP?

Dependency scanning (npm audit, Dependabot, OWASP Dependency-Check), secret scanning, SAST (Semgrep, CodeQL), container image scanning (Trivy), and Burp Suite for manual testing.

In the project: the design doc's last section.

Q64. How do you explain a security finding to a product owner?

In impact and likelihood: "any signed-in user can read every note by changing a number in the address; fixing it is a one-line owner check". Not rule ids.

In the project: each defect has such a story.

Q65. What is threat modelling, and where does QA fit?

Listing what can go wrong (assets, attackers, entry points, for example with STRIDE) before building. QA brings test ideas and checks that the mitigations are tested.

In the project: the defect catalogue is a small, concrete version: threat, category, test.

Q66. What was the most useful lesson from this project?

That a scanner and tests see different things: ZAP found every header and cookie defect and the XSS, but none of the access-control defects, and it found two weaknesses in the build I thought was done.

In the project: the verify summary shows both sides in one table.